

COURSE DETAILS

CAMPUS	MALAYSIAN INSTITUTE OF INFORMATION TECHNOLOGY
COURSE NAME	PRINCIPLES OF ARTIFICIAL INTELLIGENCE
COURSE CODE	ISB 46703
COURSE LEADER	Dr. Chen
LECTURER	Dr. Chen, Mr. Haqkiem
YEAR/SEMESTER	Jul 26

ASSESSMENT DETAILS

TITLE/NAME	COMPREHENSIVE MINI-PROJECT: MULTI-AGENT SYSTEM DESIGN
WEIGHT	25%
DATE/DEADLINE	Week 7
OUTCOME(S)	Design, orchestrate, and evaluate a Multi-Agent System (MAS) (PLO 3, PLO 6) Complete all tasks in a GROUP of TWO, THREE, or FOUR.

INSTRUCTIONS	AI-Assisted Development Policy: The use of Generative AI (e.g., ChatGPT, Claude, Cline, Copilot) to write the core application code is fully permitted and expected. Consequently, your assessment will NOT focus on manual coding syntax, but rather on Architectural Rationality, Code Robustness, Prompt Engineering Quality, and Annotation Sufficiency. Copying another group's architectural design is strictly prohibited. Mark deductions will apply for late submissions. Report and codes submission (VLE only): - Only the team leader submits to VLE before the due date. - Submissions include: - Source Code (.zip file): Compress your project environment, excluding massive virtual environments (e.g., <code>venv</code>, <code>node_modules</code>). DELIVERABLES Include <code>requirements.txt</code> or <code>Dockerfiles</code>. - Project Report (.pdf file): A comprehensive report detailing the MAS architecture, prompt designs, and evaluation results. 3. Naming Convention: <code>Project_MAS_LeaderName_LeaderStudentID.zip</code>. 4. Repository: The code must be hosted on a version control platform like GitHub. The repository link must be included in the project report. Ensure API keys are NOT pushed to the repository.
--------------	--

Section A: Application Development (50%)

1. Project Overview

Your team is tasked with designing and implementing a Multi-Agent System (MAS) utilizing orchestration frameworks such as CrewAI, LangGraph, or AutoGen.

- **Recommended Theme:** We highly recommend designing your MAS as a digital countermeasure unit against an advanced, rogue AI threat (codenamed "The Entity"). In this scenario, specialized AI agents will collaborate to analyze cyber threats, deepfakes, and formulate defenses.
- **Alternative Theme (Optional):** If your team prefers, you are permitted to select a different complex, real-world domain (e.g., Healthcare Diagnostics, Financial Market Analysis, or Scientific Research Automation), provided it meets all the architectural and technical requirements listed below.

2. Requirements for Architecture & Roles

The system must handle complex, multi-step inputs relevant to your chosen theme and process them through the following components:

- **(1) The Router (Supervisor/Dispatcher, Dynamic optional):**
 - You must implement a routing mechanism that receives the initial user prompt/data and intelligently dispatches tasks to the appropriate specialist agents.
 - *Design Requirement:* You must explicitly define whether this router is **Static** (rule/keyword-based) or **Dynamic** (LLM-driven semantic routing). A dynamic router that analyzes the intent of the input to select the workflow is more powerful but more complex.
 - *Technical consideration:* Ensure API requests from the router are mapped to the correct underlying model (e.g., avoid misrouting inference calls to embedding models like `text-embedding-nomic` by strictly managing model naming configurations in your local/remote endpoints).

- **(2) Specialized AI Agents (Minimum of 3):**

You must **clearly define the persona, system prompt, and specific tools for at least three distinct AI roles.**

If you choose the recommended theme, suggested roles include:

- *Deepfake Audio Analysis Specialist* (虚假语音识别专员)
- *Manipulated Video Stream Detector* (流视频编辑检测专员)
- *Cyber Offense/Defense Coordinator* (网络攻防专员)
- *Tactical & Strategic Predictor* (敌方战术战略预判及应对策略设计专员)

(Note: If choosing an alternative theme, you must define at least 3 similarly specialized roles for your specific domain).

- **(3) Communication & Collaboration Workflow:**

- **Define how these agents interact.** Do they **operate sequentially (Pipeline), hierarchically (Manager-Worker), or via a shared state (Blackboard/StateGraph)?**
- The workflow **must be capable of handling dependencies** (e.g., the Strategic Predictor cannot formulate a plan until the Cyber Coordinator verifies the attack vector).

- **(4) Self-Evaluation & Optimization Module (The Fallback System):**

- Once the agents generate a strategy, the system **must evaluate its own output against the initial input.**
- **Risk Mitigation:** **If an agent enters an infinite loop, hallucinates a tool, or exceeds the maximum execution time, the system must NOT crash. It must catch the error and trigger a Risk Warning and Fallback Strategy** (风险提示和退让策略), **outputting a partial response based strictly on the available processed data.**

3. Core Technical Requirements (AI-Generated Code Evaluation)

Since AI tools are expected to generate the bulk of the code, your implementation will be evaluated against the following specific, quantifiable software engineering metrics:

- **(1) Architectural Rationality & Complexity:**

- Implement at least **3 distinct agents** with mutually exclusive system prompts and responsibilities.
- Define and integrate a minimum of **3 custom tools/functions** (e.g., web search mock, data parser, file reader) that the agents can successfully invoke.
- Define at least **2 conditional routing paths** (e.g., IF threat level > 80 THEN route to Strategic Predictor, ELSE route to Standard Defense; or equivalent logic for your custom theme).

- **(2) Code Robustness & Error Handling:**

- **Timeout & Retries:** Explicitly configure `timeout` (e.g., `<= 30 seconds`) and `max_retries` (e.g., `<= 3`) limitations for all LLM API calls.
- **Exception Handling:** Implement at least **3 distinct try-except blocks** targeting specific failure points (e.g., catching `JSONDecodeError` when parsing LLM outputs, or `KeyError` in state dictionaries).
- **Fallback Execution:** The system must demonstrate a successful fallback output (partial response or error warning) in under **5 seconds** when a simulated failure (e.g., forced API disconnection or max iteration loop threshold) is triggered.

- **(3) Annotation Sufficiency:**

- **100% Docstring Coverage:** Every custom class, method, and tool function must include a standard docstring (e.g., PEP 257) explaining its purpose, inputs, and outputs.
- **Architectural Inline Comments:** A minimum of **5 substantive inline comments** must explicitly explain *why* the AI-generated logic was accepted or modified by the human developer (e.g., "Modified the AI's state update logic here to prevent overriding previous agent messages").

- **(4) Prompt Engineering Quality:**

- **Few-Shot Prompting:** At least one agent's system prompt must include a minimum of **2 explicit few-shot examples** (Input/Output pairs) to guide behavior.
- **Format Constraints:** All agents must have prompts that strictly enforce output structures (e.g., "You must output strictly in JSON format with keys: ['analysis_result', 'confidence_score', 'next_step']").

Section B: Project Report (30%)

The project report should consist of the following components:

1. **Cover Page** (Include your chosen project theme/domain).
2. **System Architecture Diagram:** A visual representation (e.g., flowcharts) of the dynamic router, agent nodes, edges, and state management.
3. **Agent Role & Prompt Design Log:** Document the system prompts designed for your 3+ agents. Include an analysis of how you refined these prompts to improve agent performance.
4. **Workflow & Communication Strategy:** Explain the collaboration pattern used and justify why it was chosen over alternatives.
5. **Robustness & Fallback Analysis:** Detail the specific fail-safes implemented for timeouts, LLM hallucinations, or routing errors.
6. **AI Usage & Code Generation Log:** A section declaring which AI tools (e.g., ChatGPT, local models via LM Studio, coding assistants) were used

- to generate the system, and how you evaluated/modified their output.
7. **Repository Link & User Guide:** Step-by-step instructions on how to install requirements and execute the MAS.
 8. **Plagiarism/AI Declaration Statement:** With all members' signatures.
 9. **References (APA format).**

Section C: Project Presentation (20%)

- Presentation must be carried out by ALL members of the group, highlighting the architectural decisions and prompt engineering strategies.
- **Live Demonstration:** You are required to run your MAS live, inputting a complex scenario relevant to your theme (e.g., a multi-vector cyber-attack), and showing the audience how the dynamic router and specialized agents communicate in real-time to produce a solution. Include a demonstration of a fallback trigger (e.g., simulating a tool failure).
- Presentation duration: 10-15 minutes.

RUBRIC

Category	Criteria	Weight
App Development (50%)	Architectural Rationality & Routing Logic: Clear implementation of the dynamic router and logical workflow design between the 3+ specialized agents.	15%
	Prompt Engineering & Role Definition: High-quality, context-rich system prompts that constrain agent behavior and ensure precise, domain-specific outputs.	15%
	Robustness & Fallback Integration: Effective error handling, parsing safeguards, loop prevention, and implementation of the risk warning/fallback strategy.	10%
	Annotation Sufficiency & Readability: Code is cleanly structured with extensive human-curated comments explaining the generated logic.	10%
Report (30%)	Architecture & Prompt Documentation: Comprehensive explanation of the MAS design, state management, and iterative prompt refinement.	15%
	Robustness Analysis & AI Declaration: Thorough documentation of edge-case handling and transparent reporting of AI generation tools used.	10%
	References & Formatting (APA Format)	5%
Presentation (20%)	Live Demo & Technical Clarity: Smooth execution of the MAS, clear explanation of the routing/collaboration process, and successful demonstration of the fallback mechanism.	20%

GOOD LUCK

The key to success is to focus on goals, not obstacles. Great things never come from comfort ZONES.